

Loan Default Risk Prediction System

Implementation of a Multilayer Perceptron from Scratch

1. Introduction and Project Overview

1.1 Project Objective

The objective of this project is to design and implement an intelligent system capable of predicting the probability of loan default using a neural network model built entirely from scratch. Unlike traditional implementations that rely on high-level deep learning frameworks (like TensorFlow or PyTorch), this project focuses on implementing the neural network manually using matrix operations. This approach demonstrates a fundamental understanding of machine learning algorithms and neural network mechanisms.

1.2 Problem Context

Financial institutions face significant losses when borrowers fail to repay loans. Accurate credit risk assessment is essential for minimizing financial risk and improving decision-making.

This system aims to assist banks by:

- Predicting client default risk.
- Supporting credit approval decisions.
- Reducing financial losses.
- Automating the risk assessment process.

1.3 System Input and Output

- **Input Data:** The system processes client features including Age, Monthly Income, Loan Amount, Loan Duration, Employment Status, and Credit History..
- **Output:** The system produces a probability of default (0 to 1) and a risk classification (Low Risk / High Risk).

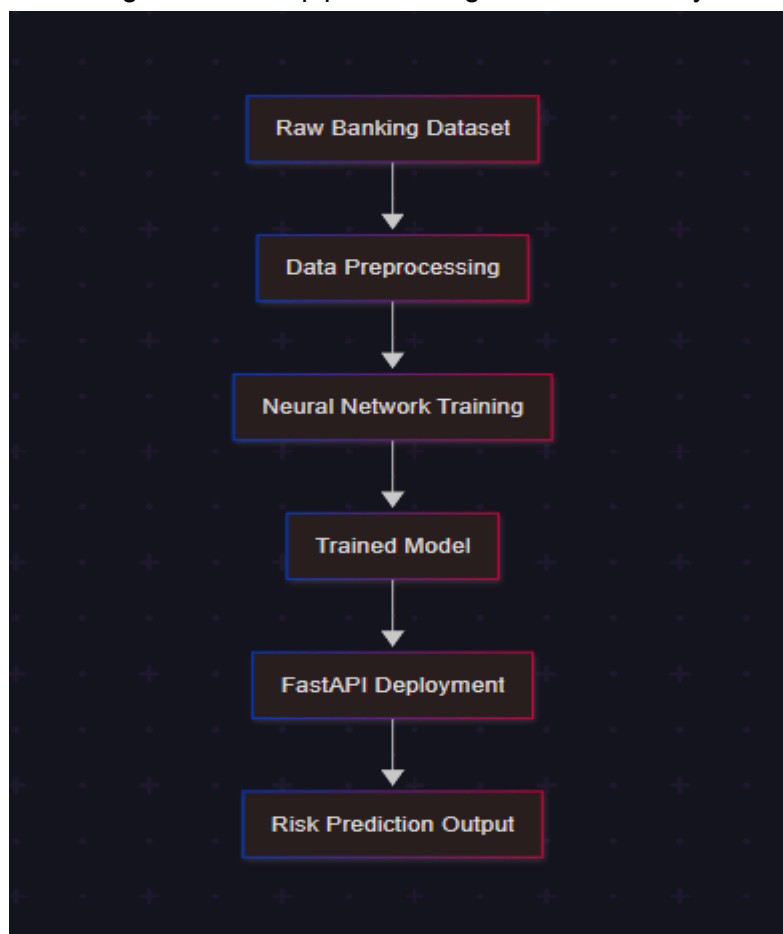
2. Technology Stack

The project will be built by manually developing each component of the neural network, utilizing a stack of lightweight tools to ensure full transparency and high computational performance.

- **Python:** Primary programming language that will be used for the neural network logic and system deployment.
- **NumPy:** Will be used for matrix operations, vectorized computation, and efficient numerical processing (The "Math Engine").
- **Pandas:** Will be used for dataset loading, data cleaning, and data transformation.
- **Matplotlib:** Will be used to visualize model performance and training progress (Loss Curves).
- **FastAPI:** Will be used to deploy the trained model as a web service.
- **Pydantic:** Will be used to ensure that user input data is valid before being processed by the model.

3. Global System Workflow

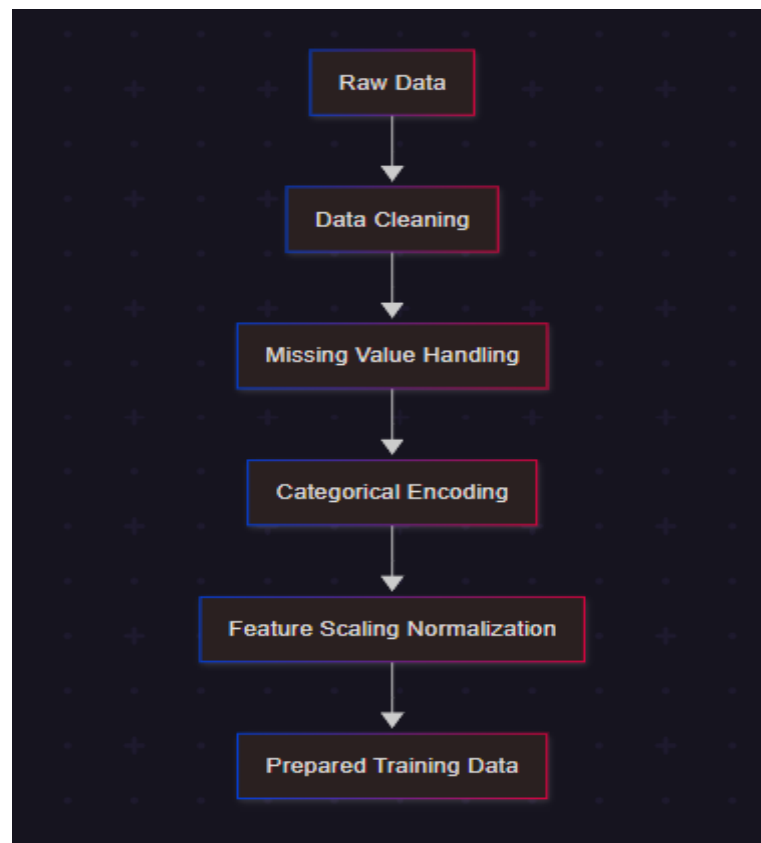
The system operates through a modular pipeline designed for scalability and maintainability.



Phase 1 — Data Preprocessing

Raw banking data must be prepared before it can enter the model:

- **Categorical Encoding:** Converting text values (e.g., "Employed") into numerical form.
- **Missing Value Handling:** Replacing empty cells using statistical methods (Mean/Median).
- **Feature Scaling:** Normalizing features between 0 and 1 so that the math remains stable.



Phase 2 — Model Training

The neural network learns from historical data. The model predicts outcomes, compares them with actual results, calculates errors, and adjusts internal parameters (Weights and Biases) to improve accuracy.

Phase 3 — Model Deployment

The trained model is saved and integrated into an API. Users submit client data via a web interface and receive predictions in real-time.

4. The Neural Network Model

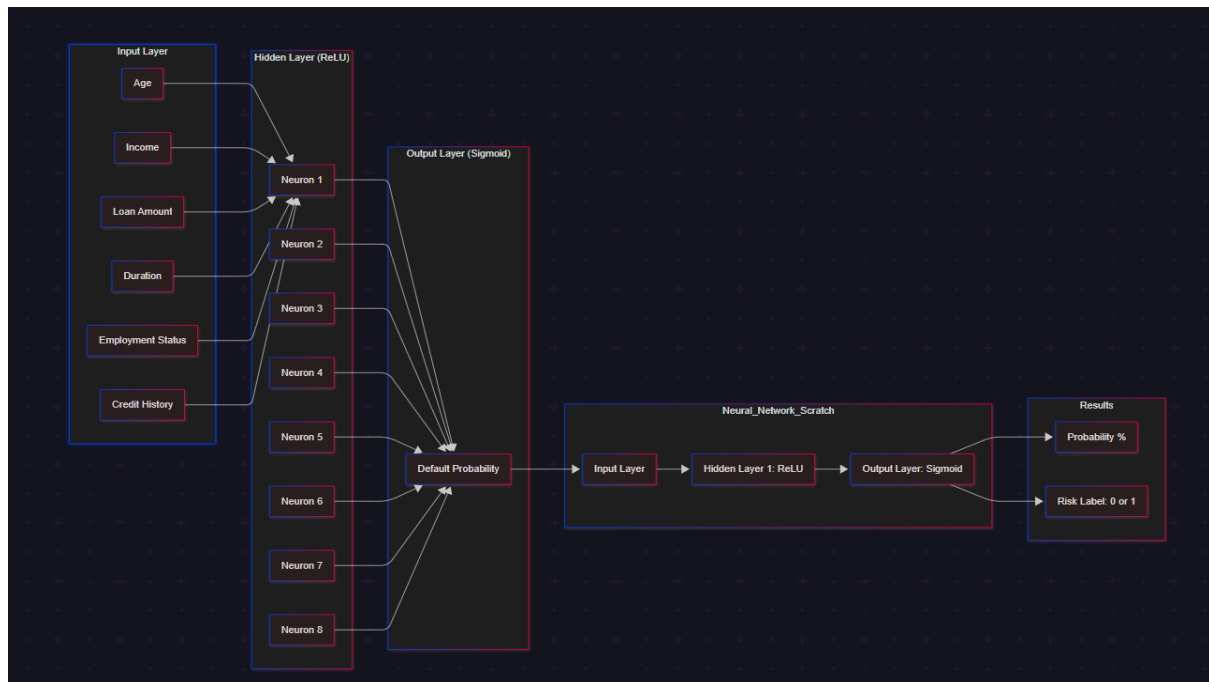
4.1 Simple Concept

A neural network is a computational model inspired by the human brain. Instead of following fixed "If-Then" rules, it learns patterns from thousands of previous examples of successful and failed loans.

4.2 Architecture

The model is a Multilayer Perceptron (MLP) consisting of:

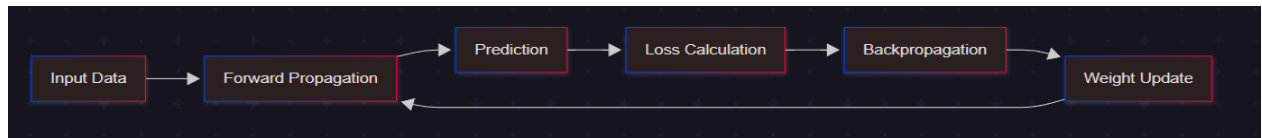
1. **Input Layer:** Receives the 6 client features.
2. **Hidden Layer (ReLU):** Identifies complex relationships between variables (e.g., how age affects the ability to pay a high loan).
3. **Output Layer (Sigmoid):** Produces a final result between 0 and 1 representing the risk.



4.3 The Learning Process (Step-by-Step)

To make the "brain" learn from scratch, I will implement a three-step cycle:

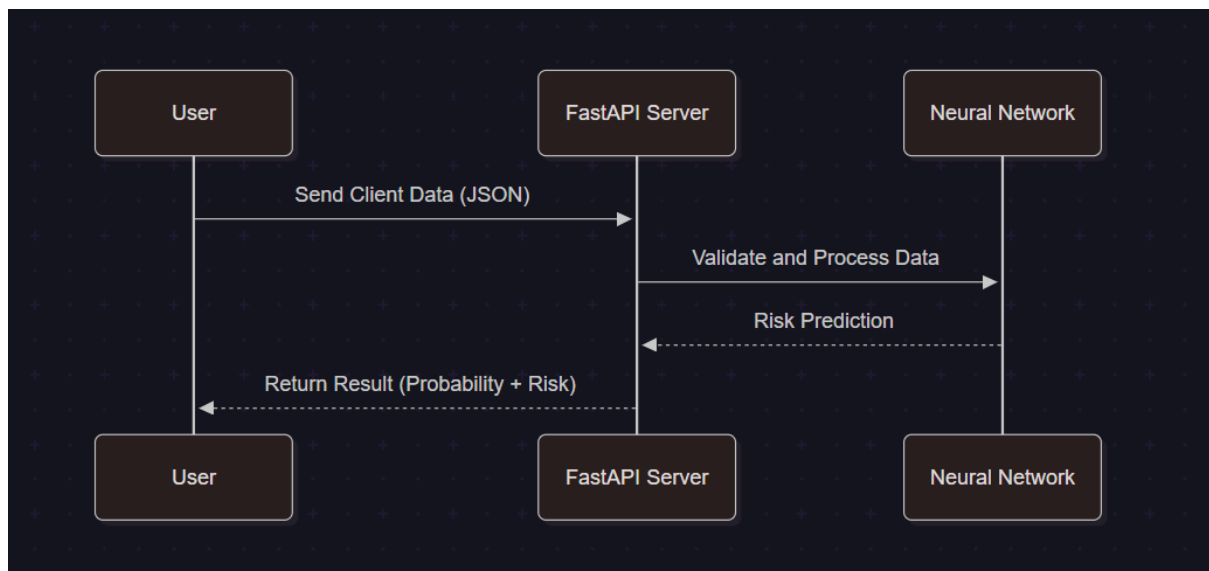
1. **Forward Propagation:** The data travels through the layers, multiplying by "Weights" to produce a guess.
2. **Loss Calculation:** The system checks how far the guess was from the actual result.
3. **Backpropagation:** The system goes backward, identifies which weights caused the mistake, and updates them so the next guess is more accurate.



5. API Prediction Workflow (Production)

Once the model is trained, it will be exposed as a REST API. This allows it to be used by banking software or mobile apps.

1. The user sends client data (JSON).
2. FastAPI validates and pre-processes the data.
3. Neural Network computes the probability.
4. The result is returned as a clear Risk Label and Probability %.



6. Conclusion

This project aims to bridge the gap between theoretical mathematics and software engineering by developing a loan risk assessment system from the ground up. I am currently implementing the neural network manually using NumPy to master the internal mechanisms of AI and ensure a deep, transparent understanding of the mathematical learning process. The final goal is to deliver a professional, efficient, and reliable risk assessment tool that is ready for real-world integration via FastAPI.

Future Work and Project Roadmap

To ensure a structured development process, the project will be completed through the following stages:

- **Document 1 — System Design (Completed):**
Presentation of the global architecture, workflow, and technology stack.
- **Document 2 — Neural Network Implementation (In Progress):**
Detailed description of the neural network architecture, training process, and implementation methodology.
- **Document 3 — Dataset and Data Engineering (In Progress):**
Description of data sources, preprocessing techniques, feature engineering, and data preparation pipeline.
- **Document 4 — Results and Evaluation (Planned):**
Model performance analysis, evaluation metrics, and system validation.